Investigating and Implementing LibreRouter and LibreMesh: Virtualized and Raspberry Pi-Based Mesh Networking

Bastas, Dimitrios (LT), Kamilos, Dimitrios (LT)
Rachel Goldstein (LT), Robert Leyba (LT)
Margaret Moore (LTJG)
EC3730 Final Project

***Abstract*—This project investigates LibreRouter and LibreMesh, open-source community networking technologies built on top of OpenWrt. A virtualized OpenWrt environment was developed using VMware Workstation Pro to simulate a four-node routed mesh backbone topology for distributed routing and mesh-network experimentation.**

Due to wireless hardware requirements within LibreMesh, OpenWrt with BATMAN-adv was utilized in a virtual environment to emulate and evaluate the same characteristics.

The project will later extend the deployment onto Raspberry Pi hardware platforms for physical mesh networking experimentation and wireless performance evaluation.

I. Introduction

Community mesh networks provide decentralized communication infrastructure capable of operating independently of centralized Internet Service Providers (ISPs). LibreRouter and LibreMesh were developed to support resilient, community-owned wireless infrastructure using open-source software and commodity hardware.

This project investigates LibreMesh deployment using both virtualized and physical implementations. Primary goals include evaluating routing behavior, deployment complexity, resilience, security considerations, and operational limitations associated with decentralized mesh infrastructure.

In addition to traditional research analysis, this report maintains a reproducible engineering notebook style documenting implementation steps, troubleshooting observations, operational anomalies, and experimental outcomes throughout the project lifecycle.

II. Background Research

A. OpenWrt

OpenWrt is an open-source Linux-based operating system designed for embedded networking devices such as routers and access points. Unlike traditional vendor firmware, OpenWrt provides a writable filesystem, modular package management, and extensive routing customization capabilities.

In addition to embedded hardware deployments, OpenWrt supports x86 virtualized environments, making it suitable for experimental mesh-networking research.

B. LibreMesh

LibreMesh is a framework built on top of OpenWrt that simplifies decentralized mesh-network deployment. The platform automates wireless mesh configuration while supporting routing protocols such as BATMAN-adv and BMX7.

LibreMesh was designed for community-operated networks in which nodes dynamically discover neighboring devices and establish resilient communication paths without centralized infrastructure.

LibreMesh includes a feature known as AnyGW (Any Gateway), which allows nodes with Internet connectivity to advertise themselves as gateways to the rest of the mesh network. Client devices use a shared virtual gateway address, while the mesh automatically selects an appropriate Internet-connected node to forward traffic toward external networks. This approach enables Internet access to be distributed throughout the mesh without requiring manual gateway configuration on each node. AnyGW improves scalability and resilience by allowing gateway services to be dynamically discovered and utilized as network conditions change. [22], [23]

C. LibreRouter

LibreRouter is an open-hardware networking platform developed specifically for community mesh deployments. The platform combines OpenWrt-based firmware with wireless hardware optimized for long-range and community-managed networking applications.

III. Literature Review and Security Analysis

A. LibreMesh and LibreRouter Deployment Model

LibreMesh is described as a modular framework for creating OpenWrt-based firmware images for wireless mesh nodes. Its goal is to simplify community mesh-network deployment by automating configuration that would otherwise require significant manual OpenWrt networking knowledge [8]. LibreMesh packages support automatic configuration of mesh behavior, while LibreRouter represents a hardware platform designed specifically for community wireless mesh networks [11].

The LibreMesh getting-started documentation recommends hardware with at least 16 MB of flash, 128 MB of RAM, one operational 2.4 GHz radio, and one operational 5 GHz radio [9]. This hardware requirement is significant because VMware

virtual machines emulate Ethernet interfaces rather than true IEEE 802.11 wireless radios. As a result, this project could validate BATMAN-adv mesh behavior over virtual Ethernet, but not full RF-based LibreMesh wireless operation.

B. BATMAN-adv Routing Literature

BATMAN-adv is a Linux kernel-based Layer 2 mesh routing protocol. Unlike traditional Layer 3 routing protocols, BATMAN-adv forwards Ethernet frames and presents participating mesh nodes as if they were connected through a distributed virtual switch [12]. OpenWrt documentation identifies batman-adv as the recommended modern BATMAN implementation for local mesh deployments [13].

BATMAN-adv uses originator messages and distributed topology information to select forwarding paths dynamically. LibreMesh documentation notes that BATMAN-adv periodically sends Originator Messages, with protocol timing affecting convergence speed and control traffic overhead [10]. This aligns with the experimental observations in this project, where node failure, recovery, and reintegration were visible through BATMAN neighbor and originator tables.

C. Community Network Case Studies

Community mesh networks have been studied and deployed in multiple real-world environments. Guifi.net is one of the most widely cited examples of a large-scale community network. Research on Guifi.net describes a crowdsourced network infrastructure operated as a shared commons and supported by community-developed tools and governance processes [16]. Additional experimental analysis of Guifi.net evaluated wireless community mesh behavior and highlighted the operational complexity of hybrid wired and wireless links [17].

Freifunk provides another major example of community mesh networking. The Gluon firmware framework, used by multiple Freifunk communities, is also based on OpenWrt and provides modular firmware generation for wireless mesh nodes [18]. This parallels LibreMesh by showing that community networks often require repeatable firmware-building tools rather than one-off manual router configuration.

NYC Mesh provides a U.S.-based example of volunteer-operated community networking. NYC Mesh uses rooftop wireless links and community-operated infrastructure to provide local broadband access, demonstrating that mesh and community-network concepts can scale beyond laboratory environments [19]. These case studies show that the technical challenges encountered in this project, including addressing, routing, management, and hardware constraints, are also present in real community-network deployments.

D. Security Weaknesses and Operational Risks

Wireless mesh networks introduce several security and operational risks. General wireless mesh security literature identifies attacks involving unauthorized access, routing disruption, malicious node participation, denial-of-service, and traffic interception [20]. Because mesh networks rely on peer participation and distributed forwarding, compromised or misconfigured nodes can affect more than one local link.

OpenWrt package management and update behavior also represent an important security area. OpenWrt security documentation notes that user-space packages can be updated relatively quickly, while kernel and kernel-module fixes may require new releases [14]. This is relevant because BATMAN-adv depends on kernel-module support, and kernel-module availability affects whether mesh routing can be deployed or patched easily.

The project also encountered package-management weaknesses during implementation. BusyBox wget lacked HTTPS support in the tested OpenWrt image, forcing temporary downgrade from HTTPS to HTTP repository access. This introduced potential risks including repository spoofing, package tampering, and man-in-the-middle attacks. OpenWrt package-signature issues have also appeared in public issue reports, demonstrating that repository trust and update-chain reliability are practical concerns rather than purely theoretical risks [15].

E. Implementation Lessons from Prior Projects

Prior mesh projects demonstrate that deployment repeatability is essential. LibreMesh, Gluon, and OpenWrt ImageBuilder workflows all emphasize firmware generation, package selection, and hardware compatibility. This supports the need for exported configuration files and repeatable setup scripts in this project.

The Commotion project provides another example of decentralized mesh-network tooling and emphasizes usability, configuration management, and security review in mesh-network deployments [21]. These lessons align with this project's experience: successful routing was only one part of the deployment; management access, configuration persistence, file transfer, script compatibility, and reproducible exports were also required to create a usable mesh-network testbed.

F. Summary of Identified Weaknesses

The literature review and project implementation identified several major weakness categories:

- Hardware dependency on supported wireless radios for full LibreMesh deployment
- Package-management and update-chain risks in embedded router systems
- Kernel-module dependency for BATMAN-adv functionality
- Potential routing disruption from malicious or misconfigured mesh nodes
- Management-plane instability caused by cloned router identities and DHCP conflicts
- IPv6 duplicate-address behavior after virtual-machine cloning
- Limited realism of virtual Ethernet compared to physical wireless RF environments
- Need for repeatable configuration export and deployment documentation

IV. Virtualized OpenWrt Deployment

A. Virtualization Environment

VMware Workstation Pro 17 was selected as the virtualization platform for the initial implementation phase. Virtualization provided rapid deployment, rollback capability through snapshots, reproducible experimentation, and simplified troubleshooting prior to deployment on physical Raspberry Pi hardware.

B. OpenWrt Image Preparation

The OpenWrt x86_64 ext4 combined image was downloaded from the official OpenWrt release repository. Since VMware does not natively boot raw IMG files efficiently, the OpenWrt disk image was converted into VMware VMDK format prior to deployment.

C. Virtual Machine Configuration

A custom virtual machine configuration was selected in VMware Workstation Pro to allow manual control over networking interfaces and storage configuration.

TABLE I
Initial OpenWrt VM Configuration

Setting	Value
Guest OS	Other Linux 5.x 64-bit
vCPUs	1
Memory	256 MB
Primary Disk	OpenWrt VMDK
Network Adapter 1	NAT
Network Adapter 2	Host-only

D. Network Interface Design

Two virtual network adapters were configured for each virtual router node:

- NAT adapter for WAN connectivity and package installation
- Host-only adapter for internal mesh backbone communication

This architecture allowed the virtual environment to simulate isolated routed mesh connectivity while preserving Internet access for package installation and updates.

E. Initial Boot Validation

The OpenWrt virtual machine booted successfully after the VMDK disk image was attached to the VMware virtual SCSI controller.

During startup, OpenWrt initialized both virtual Ethernet interfaces successfully:

```
eth0 NIC Link is Up
eth1 NIC Link is Up
```

The successful initialization confirmed that VMware virtual networking devices were correctly recognized by the OpenWrt kernel.

Fig. 1. Successful OpenWrt boot and interface initialization.

V. Network Troubleshooting and Recovery

A. Initial Connectivity Failure

Initial deployment attempts failed to provide Internet connectivity.

Observed symptoms included:

- Missing default routes
- Failed DNS resolution
- Inability to ping external IP addresses
- DHCP inconsistencies

Example failure:

```
ping -c 4 8.8.8.8
Network unreachable
```

Investigation revealed that VMware host-only networking was unintentionally acting as the WAN DHCP source, creating interface ambiguity and routing instability.

Observed VMware virtual networks included:

- VMnet1: 192.168.61.0/24
- VMnet8: 192.168.111.0/24

B. Controlled Simplification Strategy

To isolate the networking issue, the second adapter was temporarily removed and the deployment simplified to a single NAT interface.

After simplification, the OpenWrt node successfully received:

- DHCP assignment
- Default gateway
- External DNS access

Working configuration:

```
default via 192.168.111.2 dev eth0
```

Successful validation tests included:

```
ping -c 4 8.8.8.8
ping -c 4 google.com
```

All tests completed successfully with zero packet loss.

C. Package Management Failure Analysis

Package management operations initially failed during APK repository access.

Observed errors included:

```
wget exited with error 1
unexpected end of file
```

Further investigation revealed that BusyBox wget lacked HTTPS support:

```
wget https://downloads.openwrt.org/
HTTPS support not compiled in.
```

This explained repeated failures retrieving APK repository metadata over HTTPS and forced temporary downgrade of repository transport from HTTPS to HTTP in order to restore package functionality.

Potential security implications included:

- Man-in-the-middle attacks
- Package tampering
- Repository spoofing
- Supply-chain compromise

D. Temporary Package Recovery

HTTPS functionality was partially restored by manually transferring APK packages into the virtual machine and installing them locally:

```
apk add --allow-untrusted ./*.apk
```

Repository transport was later temporarily modified from HTTPS to HTTP to restore repository index access.

After modification:

```
apk update
```

successfully completed.

Core administrative packages were then installed successfully:

```
apk add nano
apk add vim
apk add curl
apk add tcpdump
apk add htop
apk add bash
apk add luci
```

VI. LuCI Web Interface Deployment

After installation of LuCI, the OpenWrt web interface was initially inaccessible through:

```
http://192.168.1.1
```

Diagnostic analysis using:

```
netstat -tlnp
```

confirmed that the web server was actively listening on ports 80 and 443:

```
0.0.0.0:80 LISTEN
0.0.0.0:443 LISTEN
```

Further analysis revealed that the Windows host system could directly access only the VMware NAT interface rather than the internal OpenWrt LAN bridge. As a result, the LuCI interface became accessible using:

```
http://192.168.111.130
```

rather than the internal bridge address.

This demonstrated the distinction between:

- Internal OpenWrt LAN addressing
- Externally reachable VMware NAT addressing
- Hypervisor network segmentation behavior

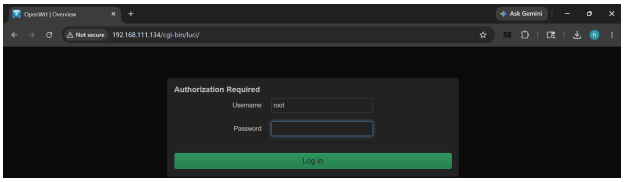

Fig. 2. Accessible LuCI web management login page

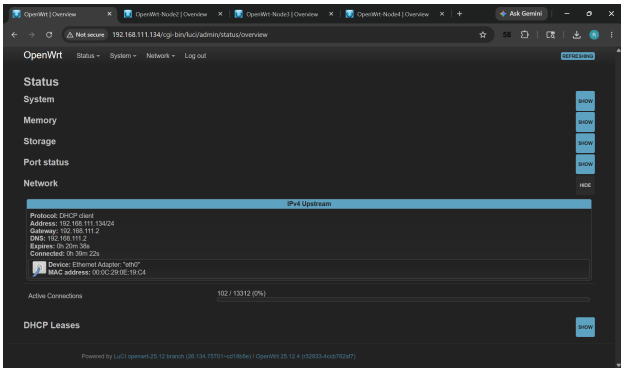

Fig. 3. Accessible LuCI web management interface through VMware NAT networking.

VII. Multi-Node Mesh Backbone Construction

A. Dual-Interface Mesh Backbone

After establishing a stable baseline, a second virtual adapter was reintroduced using VMware host-only networking.

The second interface was assigned as a dedicated mesh backbone network:

```
uci set network.mesh=interface
uci set network.mesh.proto='static'
uci set network.mesh.device='eth1'
uci set network.mesh.ipaddr='10.10.10.1'
uci set network.mesh.netmask='255.255.255.0'
uci commit network
/etc/init.d/network restart
```

Resulting topology:

TABLE II
Dual-Interface OpenWrt Topology

Interface	Address	Purpose
eth0	192.168.111.x	VMware NAT / WAN
eth1	10.10.10.x	Internal mesh backbone

Routing verification confirmed correct segmentation:

```
default via 192.168.111.2 dev eth0
10.10.10.0/24 dev eth1
```

```
root@OpenWrt:~# ip route
default via 192.168.111.2 dev eth0  src 192.168.111.134
10.10.10.0/24 dev eth1 scope link  src 10.10.10.1
192.168.1.0/24 dev eth0 scope link  src 192.168.1.1
192.168.111.0/24 dev eth0 scope link  src 192.168.111.134
root@OpenWrt:~# _
```

Fig. 4. Validated OpenWrt Node 1 routing table showing WAN and mesh backbone separation.

B. VM Cloning Strategy

Instead of reinstalling OpenWrt repeatedly, a stable "golden image" virtual machine was cloned using VMware full-clone functionality.

Resulting nodes included:

- OpenWrt-Node1
- OpenWrt-Node2
- OpenWrt-Node3
- OpenWrt-Node4

Each node received:

- Unique hostname
- Unique mesh backbone address
- VMware-generated MAC address

C. Four-Node Mesh Backbone Validation

The resulting mesh backbone addressing scheme included:

TABLE III
FOUR-NODE MESH BACKBONE ADDRESSING

Node	Mesh Address
OpenWrt-Node1	10.10.10.1
OpenWrt-Node2	10.10.10.2
OpenWrt-Node3	10.10.10.3
OpenWrt-Node4	10.10.10.4

Connectivity testing was performed between all nodes using ICMP echo requests.

Successful bidirectional communication was verified between all nodes with:

- Zero packet loss
- Stable latency
- Successful route propagation

The successful validation demonstrated:

- Stable Layer 2 adjacency
- Functional Layer 3 routing
- Operational VMware host-only backbone networking
- Successful multi-node OpenWrt virtualization

```
root@OpenWrt-Node4:~# ping -c 2 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=2.18 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=1.33 ms

--- 10.10.10.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1004ms
rtt min/avg/max/mdev = 1.334/1.755/2.176/0.421 ms
root@OpenWrt-Node4:~# ping -c 2 10.10.10.2
PING 10.10.10.2 (10.10.10.2) 56(84) bytes of data.
64 bytes from 10.10.10.2: icmp_seq=1 ttl=64 time=2.23 ms
64 bytes from 10.10.10.2: icmp_seq=2 ttl=64 time=4.85 ms

--- 10.10.10.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 2.234/3.542/4.851/1.308 ms
root@OpenWrt-Node4:~# ping -c 2 10.10.10.3
PING 10.10.10.3 (10.10.10.3) 56(84) bytes of data.
64 bytes from 10.10.10.3: icmp_seq=1 ttl=64 time=1.92 ms
64 bytes from 10.10.10.3: icmp_seq=2 ttl=64 time=1.48 ms

--- 10.10.10.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 1.480/1.700/1.921/0.220 ms
root@OpenWrt-Node4:~#
```

Fig. 5. Successful four-node mesh backbone connectivity validation.

D. Sequential Startup and DHCP Convergence

Initial simultaneous startup of cloned nodes produced temporary WAN IP duplication behavior within the VMware NAT network.

Multiple nodes briefly inherited identical DHCP leases from previous cloned states.

This behavior was mitigated by sequential node startup. Sequential startup allowed VMware DHCP services to converge properly and assign unique WAN addresses based on MAC address differentiation.

Final WAN assignments stabilized as:

- Node1: 192.168.111.134
- Node2: 192.168.111.136
- Node3: 192.168.111.137
- Node4: 192.168.111.138

This demonstrated:

- DHCP lease convergence behavior
- Temporary ARP instability during cloned startup
- Virtualization timing dependencies
- Successful MAC-based lease differentiation

E. IPv6 Duplicate Address Detection Behavior

During sequential startup of cloned nodes, OpenWrt repeatedly reported IPv6 duplicate address detection warnings:

```
IPv6 duplicate address detected
```

Despite repeated IPv6 duplicate detection events:

- IPv4 routing remained stable
- Mesh backbone communication remained operational
- Node-to-node connectivity was unaffected

This behavior represents an important operational consideration for cloned embedded router deployments and virtualized distributed networking systems.

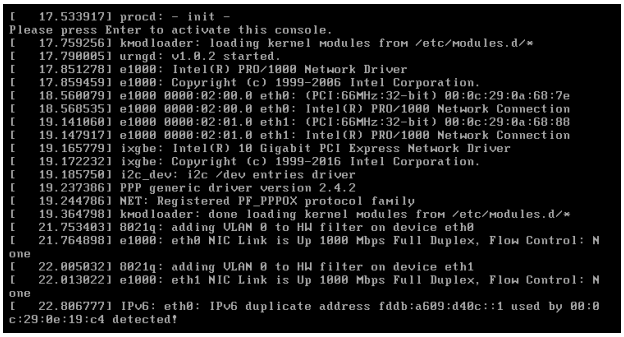

Fig. 6. Observed IPv6 duplicate address detection during sequential startup of cloned OpenWrt nodes.

VIII. BATMAN-adv Mesh Routing Deployment

A. BATMAN-adv Installation

After the successful construction of the four-node OpenWrt backbone topology, the BATMAN-adv routing protocol was deployed to provide dynamic Layer 2 mesh routing functionality.

The following packages were installed on all nodes:

```
apk add kmod-batman-adv batctl luci-proto-batman-adv
```

Successful installation loaded the BATMAN kernel module dynamically:

```
lsmod | grep batman
```

The BATMAN mesh interface was manually initialized:

```
modprobe batman-adv
ip link add name bat0 type batadv
ip link set up dev bat0
```

The dedicated mesh transport interface was attached to BATMAN:

```
batctl meshif bat0 if add eth1
```

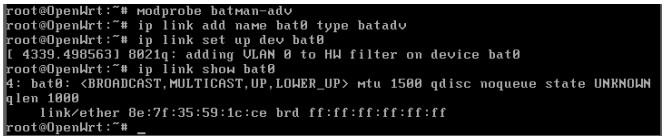

Fig. 7. Successful initialization of the BATMAN-adv virtual mesh interface.

B. Dynamic Neighbor Discovery

BATMAN dynamically discovered neighboring routers using:

```
batctl meshif bat0 n
```

Originator propagation tables were observed using:

```
batctl meshif bat0 o
```

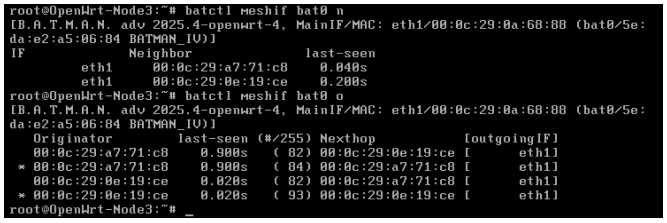

Fig. 8. BATMAN-adv originator routing tables dynamically populated across the mesh.

C. Four-Node Mesh Convergence

All four OpenWrt nodes successfully joined the distributed BATMAN mesh.

The resulting deployment demonstrated:

- Autonomous neighbor discovery
- Dynamic route propagation
- Persistent distributed convergence
- Self-healing mesh behavior

Fig. 9. Stable four-node BATMAN-adv mesh topology before failure testing.

D. Node Failure and Reconvergence

One node was intentionally powered off during operation to evaluate mesh resilience behavior.

Immediately after failure:

- Neighbor tables updated dynamically
- Stale originators were removed
- Remaining nodes maintained connectivity

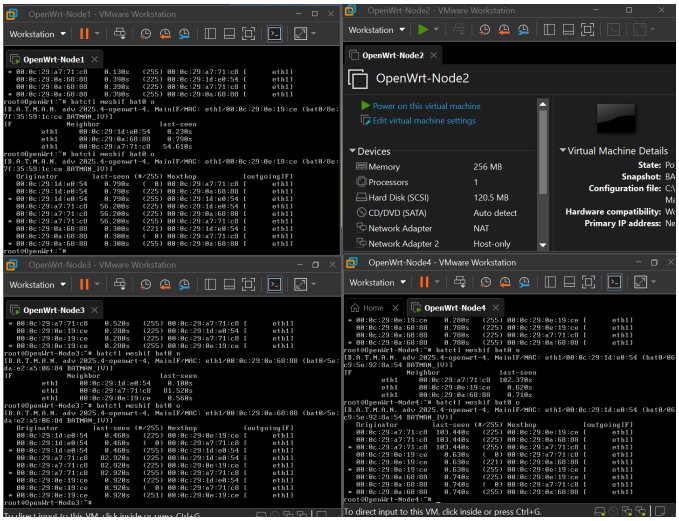

Fig. 10. BATMAN topology immediately following intentional node failure.

E. Node Reintegration

After reboot, the failed node automatically rejoined the distributed mesh without manual reconfiguration.

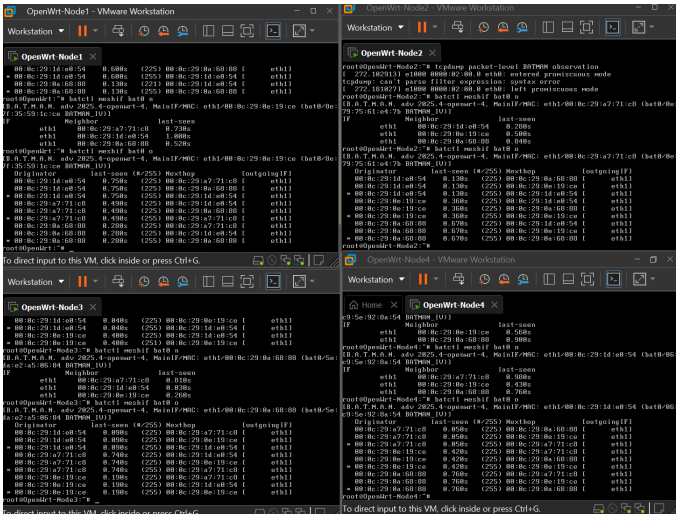

Fig. 11. Automatic BATMAN mesh reconvergence after node restoration.

F. Persistent BATMAN Configuration

Persistent deployment was implemented using OpenWrt UCI configuration:

```
uci set network.bat0=interface
uci set network.bat0.proto='batadv'
uci set network.bat0.routing_algo='BATMAN_IV'

uci set network.mesh_bat=interface
uci set network.mesh_bat.proto='batadv_hardif'
uci set network.mesh_bat.master='bat0'
uci set network.mesh_bat.device='eth1'

uci commit network
```

After reboot, all nodes automatically:

- Recreated bat0
- Reattached eth1
- Rediscovered neighbors
- Rejoined the mesh

G. BATMAN Native Ping and Traceroute

BATMAN-native connectivity was validated using:

```
batctl meshif bat0 ping <neighbor MAC>
```

Traceroute functionality was validated using:

```
batctl meshif bat0 traceroute <neighbor MAC>
```

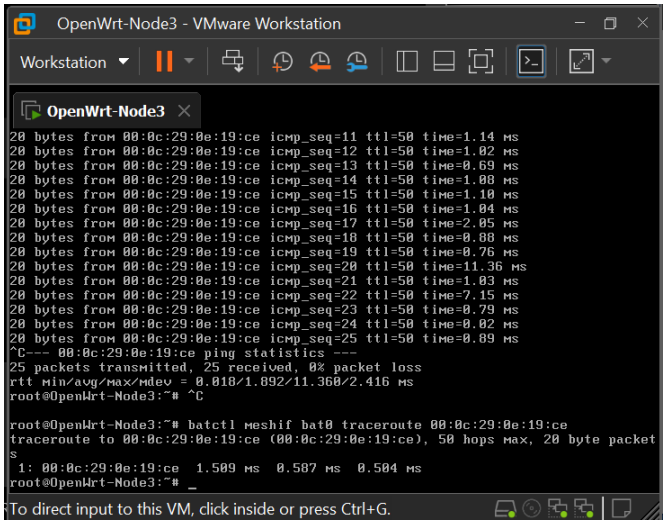

Fig. 12. BATMAN-native traceroute across the distributed mesh topology.

H. MTU Optimization and Fragmentation Analysis

BATMAN generated warnings indicating insufficient MTU allocation on the transport interface.

Initial configuration:

- eth1 MTU: 1500
- bat0 MTU: 1500

The transport interface MTU was increased experimentally:

```
ip link set dev eth1 mtu 1532
```

Following optimization:

- Neighbor discovery remained stable
- BATMAN forwarding persisted
- Packet loss remained zero

Interestingly, bat0 continued operating at MTU 1500 while eth1 operated at MTU 1532.

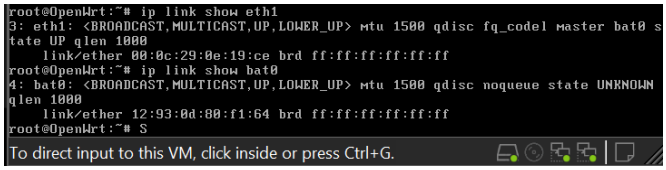

Fig. 13. Initial BATMAN deployment with insufficient MTU allocation.

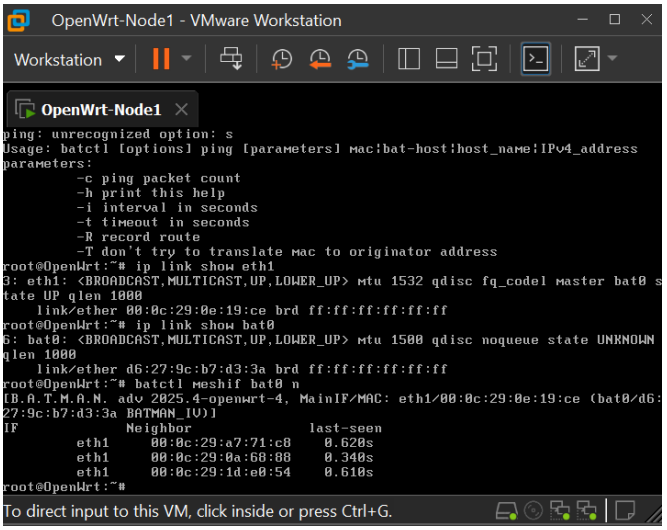

Fig. 14. Stable BATMAN operation following MTU optimization.

I. Management-Plane Addressing Issues

During virtual-machine cloning, multiple nodes temporarily inherited overlapping management-plane identities.

Observed effects included:

- Duplicate DHCP assignments
- Ambiguous LuCI access
- IPv6 duplicate-address warnings

Despite management-plane ambiguity, the BATMAN forwarding plane remained fully operational.

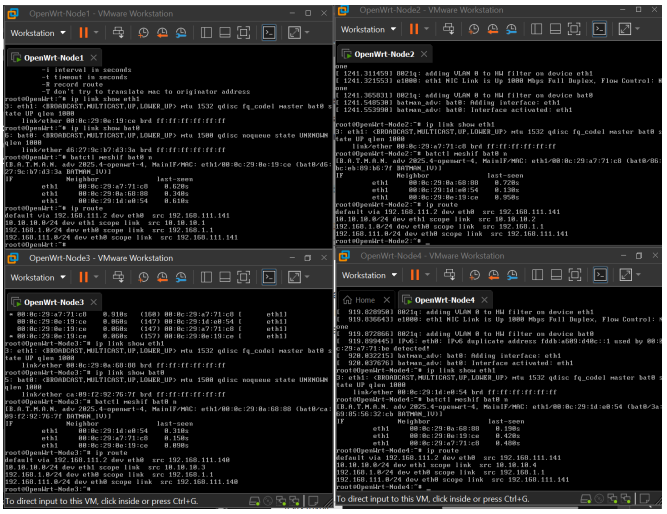

Fig. 15. Management-plane addressing conflicts during cloned OpenWrt deployment.

IX. LibreMesh Firmware and Wireless Hardware Requirements

The LibreMesh project provides prebuilt OpenWrt-based firmware images designed specifically for decentralized wireless mesh deployments.

Installation procedures and firmware selection guidance were obtained from the official LibreMesh documentation and firmware selector platform.

LibreMesh installation generally requires:

- OpenWrt-supported hardware
- At least 16 MB flash storage
- At least 128 MB RAM
- One operational 2.4 GHz wireless radio
- One operational 5 GHz wireless radio

The dual-radio recommendation exists because LibreMesh commonly separates:

- Client-access traffic
- Mesh-backhaul traffic

Typical deployments utilize:

- 2.4 GHz radios for local client access and extended range
- 5 GHz radios for higher-throughput mesh backhaul links

LibreMesh firmware may be obtained using:

- LibreMesh Firmware Selector
- Precompiled firmware archives
- Custom OpenWrt ImageBuilder compilation

The project initially investigated whether full LibreMesh wireless meshing could be reproduced entirely inside VMware virtualization.

However, VMware virtual network adapters emulate Ethernet devices rather than physical IEEE 802.11 wireless radios.

As a result, VMware virtual adapters do not provide:

- True wireless PHY behavior
- RF channel negotiation
- Wireless beaconing
- 802.11s mesh operation
- Ad-hoc wireless functionality
- Monitor-mode radio capabilities

Although BATMAN-adv operated successfully over virtual Ethernet interfaces, this represented a Layer 2 routed mesh simulation rather than a true wireless LibreMesh deployment.

The virtualized environment therefore successfully validated:

- Distributed BATMAN routing behavior
- Dynamic neighbor discovery
- Autonomous route propagation
- Persistent distributed convergence

However, full LibreMesh wireless operation will require physical wireless hardware during the Raspberry Pi deployment phase.

X. Configuration Export and Deployment Reproducibility

To satisfy reproducibility requirements, configuration artifacts and operational mesh-state information were exported directly from all OpenWrt nodes.

The exported artifacts included:

- Persistent OpenWrt UCI configuration files
- BATMAN neighbor tables
- BATMAN originator tables

- Routing tables
- Interface-state information
- Installed package inventories

A reusable export automation script was developed to simplify artifact extraction across all nodes.

The script automatically:

- Collected configuration files
- Captured BATMAN operational state
- Archived deployment artifacts
- Generated compressed export bundles

The resulting deployment archives were extracted remotely from the OpenWrt nodes using SSH and SCP.

A. Automated Configuration Export Script

The following script was deployed on all OpenWrt nodes:

```
#!/bin/sh

NODE_NAME="$(cat /proc/sys/kernel/hostname)"
OUTDIR="/root/config-export"
ARCHIVE="/root/${NODE_NAME}-config-export.tar.gz"

mkdir -p "$OUTDIR"

cp /etc/config/network \
"$OUTDIR/network-${NODE_NAME}"

cp /etc/config/system \
"$OUTDIR/system-${NODE_NAME}"

cp /etc/config/firewall \
"$OUTDIR/firewall-${NODE_NAME}"

cp /etc/config/dhcp \
"$OUTDIR/dhcp-${NODE_NAME}"

batctl meshif bat0 if > \
"$OUTDIR/batman-if-${NODE_NAME}.txt"

batctl meshif bat0 n > \
"$OUTDIR/batman-neighbors-${NODE_NAME}.txt"

batctl meshif bat0 o > \
"$OUTDIR/batman-originators-${NODE_NAME}.txt"

ip addr > \
"$OUTDIR/ip-addr-${NODE_NAME}.txt"

ip route > \
"$OUTDIR/ip-route-${NODE_NAME}.txt"

apk list --installed > \
"$OUTDIR/packages-${NODE_NAME}.txt"

tar -czf "$ARCHIVE" \
-C /root config-export
```

B. Windows and Linux Script Compatibility Issues

During deployment automation, shell scripts transferred from Windows systems to OpenWrt routers initially failed with:

```
-ash: /root/export-config.sh: not found
```

Investigation revealed that the script contained Windows CRLF line endings rather than Linux LF line endings.

This caused BusyBox ash to misinterpret the script interpreter line:

```
#!/bin/shˆM
```

The issue was corrected using:

```
sed -i 's/\r$//' /root/export-config.sh
```

This represented an important interoperability consideration between Windows administrative systems and embedded Linux networking environments.

C. SSH and SCP Artifact Extraction

Configuration archives were remotely extracted from OpenWrt nodes using SCP over SSH.

Modern Windows SCP implementations attempted to default to SFTP mode, which was unsupported by the lightweight OpenWrt Dropbear deployment.

Observed error:

```
subsystem request failed on channel 0
scp: Connection closed
```

Successful extraction required forcing legacy SCP protocol mode:

```
scp -O root@192.168.111.x:/root/archive.tar.gz .
```

This demonstrated another operational limitation common within lightweight embedded Linux distributions where full SFTP subsystems are often omitted to conserve storage space and memory resources.

XI. Git-Based Collaborative Configuration Management

To improve reproducibility, collaboration, and long-term configuration management, the project artifacts were integrated into a Git-based version-control workflow using GitHub.

The repository centralized:

- LaTeX report source files
- Figures and screenshots
- OpenWrt configuration exports
- BATMAN operational logs
- Deployment scripts
- Supporting documentation

This approach allowed multiple project participants to:

- Retrieve synchronized project files
- Contribute modifications collaboratively
- Maintain deployment history
- Track configuration revisions
- Restore previous project states if necessary

A. Repository Initialization

A new local Git repository was initialized within the project directory:

```
git init
```

A `.gitignore` file was created to exclude temporary LaTeX compilation artifacts including:

```
*.aux
*.log
*.out
*.toc
*.synctex.gz
```

Project files were staged and committed locally:

```
git add .
git commit -m "Initial LibreMesh project upload"
```

The local repository was then connected to a remote GitHub repository:

```
git remote add origin \
https://github.com/<user>/<repo>.git
```

The repository was uploaded using:

```
git branch -M main
git push -u origin main
```

B. Repository Structure

The resulting repository structure organized project artifacts into separate directories:

```
EC3730-Pro/
|-archive/
|-report/
|-virtual machines/
|-EC3730_Final_Project.pdf
|-README.md
```

This structure separated:

- Documentation
- Deployment scripts
- Configuration archives
- Experimental artifacts

C. Collaborative Workflow

Additional project participants could retrieve the repository using:

```
git clone \
https://github.com/<user>/<repo>.git
```

Once cloned locally, collaborators could synchronize project updates using:

```
git pull
```

Modified files could then be uploaded back to the shared repository using:

```
git add .
git commit -m "Describe changes"
git push
```

This created a distributed collaborative workflow allowing all participants to maintain synchronized copies of the deployment environment and report documentation.

D. Version-Control Benefits

Git-based version control provided several operational advantages during experimentation:

- Distributed backup of project artifacts
- Historical tracking of configuration changes
- Simplified collaboration between participants
- Recovery from accidental file modification
- Improved reproducibility of deployment procedures

The integration of Git-based workflow management significantly improved the maintainability and reproducibility of the overall mesh-network deployment project.

XII. Raspberry Pi Deployment

A. Custom LibreMesh Firmware Image Generation

To simplify deployment onto Raspberry Pi 5 hardware, a custom LibreMesh firmware image was generated using the LibreMesh firmware builder.

The firmware image targeted the Raspberry Pi 5 platform:

- Platform: bcm27xx/bcm2712
- Device: Raspberry Pi 5
- OpenWrt Version: 25.12.4
- LibreMesh Feed Branch: master
- Routing Protocol: BATMAN-adv

Additional management and diagnostic packages were included during image generation:

- BATMAN-adv kernel modules
- LuCI web administration interface
- SSH services
- TCPDump
- iPerf3
- Curl
- Wget
- Nano
- Htop
- JSON query utilities

The LibreMesh image builder also supports execution of a first-boot script through the OpenWrt `uci-defaults` mechanism. This allows nodes to self-configure immediately after their initial boot sequence.

B. Automated First-Boot Provisioning

A custom first-boot script named:

`EC3730_firstboot_router_ap`

was developed to automatically provision a Raspberry Pi LibreMesh node.

The script performs the following actions:

- Sets a predefined hostname
- Creates centralized log directories
- Verifies installed package availability
- Configures a common EC3730 wireless network
- Enables WPA2 wireless security
- Removes stale client-mode wireless configurations
- Preserves LibreMesh BATMAN-adv operation
- Adjusts DHCP address allocation behavior
- Creates mesh troubleshooting utilities
- Creates packet-capture utilities
- Creates automated connectivity testing utilities
- Generates deployment logs

The deployment standardizes all classroom nodes using:

SSID: EC3730
Password: EC3730Mesh!

To improve address-management consistency, the DHCP pool is configured to begin allocating client leases at host offset 11. This reserves the first ten low host addresses for future infrastructure assignments and static device allocation.

The provisioning process also generates several operational support tools:

- `check_mesh.sh`
- `capture_mesh.sh`
- `test_connectivity.sh`

These scripts automate routine troubleshooting tasks including:

- BATMAN neighbor verification
- BATMAN originator analysis
- Wireless status verification
- Routing-table inspection
- Packet capture generation
- Connectivity testing

Following firmware flashing and initial boot, a Raspberry Pi node can automatically transition into a managed LibreMesh deployment without requiring extensive manual configuration.

This approach significantly reduces deployment time, improves reproducibility, and minimizes configuration drift between participating mesh-network nodes.

C. Firmware Flashing and Initial Boot

After generating the customized LibreMesh firmware image, the image was written to a microSD card using the Rufus imaging utility.

Rufus is a Windows-based USB and disk imaging tool capable of writing raw operating-system images directly to removable storage devices. The utility was selected because of its simplicity, reliability, and compatibility with OpenWrt firmware images.

The deployment procedure consisted of the following steps:

1) Download the customized LibreMesh/OpenWrt firmware image.
2) Insert a microSD card into the deployment workstation.
3) Launch Rufus.
4) Select the target microSD card.
5) Select the LibreMesh firmware image.
6) Write the image to the microSD card.
7) Insert the microSD card into the Raspberry Pi 5.
8) Apply power and allow the initial boot process to complete.

Following successful boot, the Raspberry Pi automatically initialized the LibreMesh software stack and wireless services.

Verification of successful deployment was performed by observing the appearance of the configured wireless network SSID. The node began broadcasting the deployment SSID:

EC3730

Detection of the wireless network confirmed:

- Successful firmware flashing
- Successful Raspberry Pi boot sequence
- Proper wireless-radio initialization
- Execution of LibreMesh configuration services
- Activation of the wireless access-point interface

Once the wireless network became visible, management access could be established through either:

- Direct Ethernet connection
- Secure Shell (SSH)
- LuCI web interface

The appearance of the wireless SSID provided a simple operational validation that the device had transitioned from a freshly flashed system into an operational LibreMesh node.

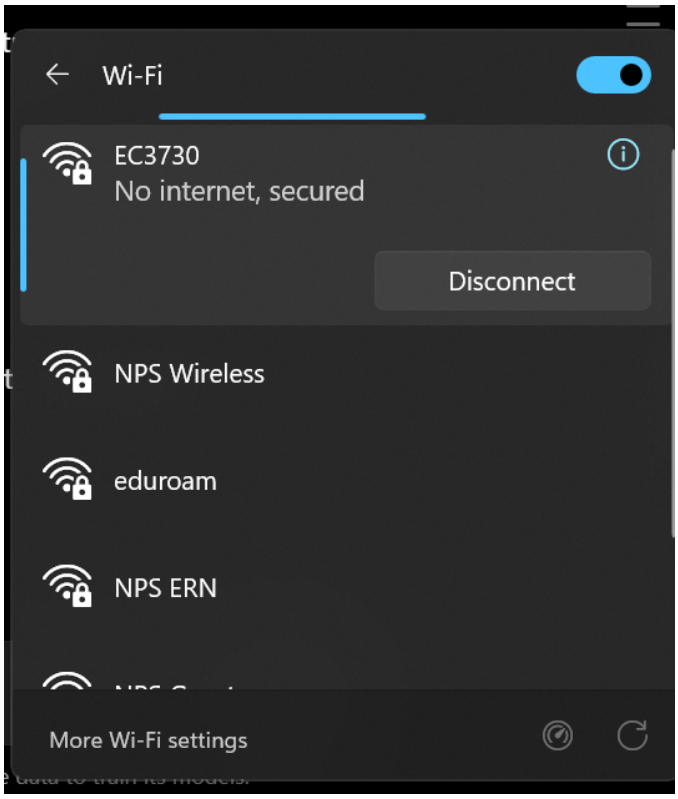

Fig. 16. Raspberry Pi 5 broadcasting the EC3730 LibreMesh wireless network following initial boot.

D. Exploring the Firmware

Following successful deployment of the customized LibreMesh firmware image onto the Raspberry Pi 5, several verification and discovery procedures were performed to better understand the default LibreMesh configuration and confirm proper system operation.

The purpose of this exploration phase was to:

- Verify successful firmware installation
- Confirm LibreMesh services were operational
- Identify automatically generated network interfaces
- Examine addressing behavior
- Validate wireless-radio functionality
- Investigate BATMAN-adv integration
- Establish a baseline configuration before further modifications

1) Routing Table Inspection: The first step was examination of the routing table using:

```
ip route
```

The routing table revealed that LibreMesh automatically configured the node within the 10.13.0.0/16 network space and created an AnyGW interface for future gateway discovery and Internet-sharing functionality.

```
root@AR1:~/ec3730/scripts# ip route
10.13.0.0/16 dev br-lan proto kernel scope link src 10.13.99.87
10.13.0.0/16 dev anygw proto static scope link metric 2147483647
root@AR1:~/ec3730/scripts#
```

Fig. 17. LibreMesh routing table after initial boot.

Unlike the VMware BATMAN deployment, where addressing and routing were configured manually, LibreMesh automatically generated the routing configuration during the first boot sequence.

2) Network Interface Analysis: Network interfaces were examined using:

```
ip addr
```

This inspection identified several automatically generated interfaces including:

- br-lan
- anygw
- bat0
- wlan0-ap

The appearance of the `bat0` interface confirmed that BATMAN-adv had been initialized automatically by LibreMesh.

```
root@AR1:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
       valid_lft forever preferred_lft forever
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel master br-lan state DOWN group default qlen 1000
    link/ether 2c:cf:67:2b:63:57 brd ff:ff:ff:ff:ff:ff
3: eth1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel master br-lan state DOWN group default qlen 1000
    link/ether 1a:c2:43:6c:62:b3 brd ff:ff:ff:ff:ff:ff
4: wlan0-ap: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master br-lan state UP group default qlen 1000
    link/ether 2e:cf:67:2b:63:59 brd ff:ff:ff:ff:ff:ff permaddr 2c:cf:67:2b:63:58
5: br-lan: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 1a:c2:43:6c:62:b3 brd ff:ff:ff:ff:ff:ff
    inet 10.13.99.87/16 brd 10.13.255.255 scope global br-lan
       valid_lft forever preferred_lft forever
    inet6 fd0d:fe46:8ce8::2b:6357/64 scope global
       valid_lft forever preferred_lft forever
    inet6 fe80::10c2:43ff:fe6c:62b3/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
6: anygw@br-lan: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether aa:aa:aa:0d:fe:aa brd ff:ff:ff:ff:ff:ff
    inet 10.13.0.1/16 brd 10.13.255.255 scope global anygw
       valid_lft forever preferred_lft forever
    inet6 fd0d:fe46:8ce8::1/64 scope global
       valid_lft forever preferred_lft forever
    inet6 fe80::a8aa:aaff:fe0d:feaa/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
7: eth0_17@eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1496 qdisc noqueue state LOWERLAYERDOWN group default qlen 1000
    link/ether 2c:cf:67:2b:63:57 brd ff:ff:ff:ff:ff:ff
    inet 10.13.99.87/32 brd 255.255.255.255 scope global eth0_17
       valid_lft forever preferred_lft forever
8: eth0_29@eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1496 qdisc noqueue state LOWERLAYERDOWN group default qlen 1000
    link/ether 02:95:39:2b:63:57 brd ff:ff:ff:ff:ff:ff
9: eth1_17@eth1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1496 qdisc noqueue state LOWERLAYERDOWN group default qlen 1000
    link/ether 1a:c2:43:6c:62:b3 brd ff:ff:ff:ff:ff:ff
```

Fig. 18. Network interfaces generated automatically by LibreMesh.

```
9: eth1_17@eth1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1496 qdisc noqueue state LOWERLAYERDOWN group default qlen 1000
    link/ether 1a:c2:43:6c:62:b3 brd ff:ff:ff:ff:ff:ff
    inet 10.13.99.87/32 brd 255.255.255.255 scope global eth1_17
       valid_lft forever preferred_lft forever
10: eth1_29@eth1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1496 qdisc noqueue state LOWERLAYERDOWN group default qlen 1000
    link/ether 02:bb:ed:2b:63:57 brd ff:ff:ff:ff:ff:ff
11: bat0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-lan state UNKNOWN group default qlen 1000
    link/ether a6:d7:7a:80:87:09 brd ff:ff:ff:ff:ff:ff
root@AR1:~# iw dev
phy#0
        Interface wlan0-ap
                ifindex 4
                wdev 0x1
                addr 2e:cf:67:2b:63:59
                ssid EC3730
                type AP
                channel 36 (5180 MHz), width: 40 MHz, center1: 5190 MHz
                txpower 31.00 dBm
root@AR1:~# lime-query network
-ash: lime-query: not found
root@AR1:~#
```

Fig. 19. Network interfaces generated automatically by LibreMesh.

The node received the IPv4 address:

```
10.13.99.87/16
```

which differs significantly from the manually assigned addresses used during the virtualized OpenWrt mesh-network deployment.

3) Wireless Interface Verification: Wireless functionality was verified using:

```
iw dev
```

Results showed that LibreMesh successfully configured the Raspberry Pi wireless subsystem and created an access-point interface.

The configured deployment SSID:

```
EC3730
```

was actively broadcast on channel 36 within the 5 GHz frequency band.

```
root@AR1:~/ec3730/scripts# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel master br-lan state DOWN mode DEFAULT group default qlen 1000
    link/ether 2c:cf:67:2b:63:57 brd ff:ff:ff:ff:ff:ff
3: eth1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel master br-lan state DOWN mode DEFAULT group default qlen 1000
    link/ether 1a:c2:43:6c:62:b3 brd ff:ff:ff:ff:ff:ff
4: wlan0-ap: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master br-lan state UP mode DEFAULT group default qlen 1000
    link/ether 2e:cf:67:2b:63:59 brd ff:ff:ff:ff:ff:ff permaddr 2c:cf:67:2b:63:58
5: br-lan: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group default qlen 1000
    link/ether 1a:c2:43:6c:62:b3 brd ff:ff:ff:ff:ff:ff
6: anygw@br-lan: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group default qlen 1000
    link/ether aa:aa:aa:0d:fe:aa brd ff:ff:ff:ff:ff:ff
7: eth0_17@eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1496 qdisc noqueue state LOWERLAYERDOWN mode DEFAULT group default qlen 1000
    link/ether 2c:cf:67:2b:63:57 brd ff:ff:ff:ff:ff:ff
8: eth0_29@eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1496 qdisc noqueue state LOWERLAYERDOWN mode DEFAULT group default qlen 1000
    link/ether 02:95:39:2b:63:57 brd ff:ff:ff:ff:ff:ff
9: eth1_17@eth1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1496 qdisc noqueue state LOWERLAYERDOWN mode DEFAULT group default qlen 1000
    link/ether 1a:c2:43:6c:62:b3 brd ff:ff:ff:ff:ff:ff
10: eth1_29@eth1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1496 qdisc noqueue state LOWERLAYERDOWN mode DEFAULT group default qlen 100
0
    link/ether 02:bb:ed:2b:63:57 brd ff:ff:ff:ff:ff:ff
11: bat0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-lan state UNKNOWN mode DEFAULT group default qlen 1000
```

Fig. 20. Wireless interface configuration and SSID verification.

This confirmed successful initialization of the onboard Raspberry Pi wireless hardware and demonstrated that the custom deployment parameters were correctly applied during image generation.

4) BATMAN-adv Verification: BATMAN-adv status commands were executed using:

```
batctl if
batctl n
batctl o
```

At the time of testing, these commands produced no neighbor or originator entries.

This behavior was expected because only a single LibreMesh node had been deployed. BATMAN-adv requires multiple participating nodes before neighbor discovery and route propagation can occur.

The presence of the `bat0` interface nevertheless confirmed that BATMAN-adv had been loaded and initialized successfully.

5) LibreMesh Management Interface: The LibreMesh management interface was accessed through a web browser using the node management address.

The management dashboard provided operational information including:

- Device model
- Firmware version
- Network status
- Assigned addresses

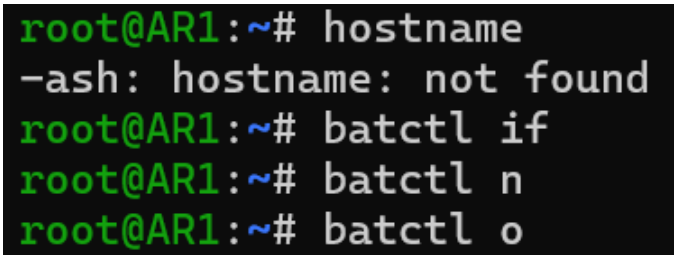

Fig. 21. BATMAN-adv verification commands executed on the Raspberry Pi node.

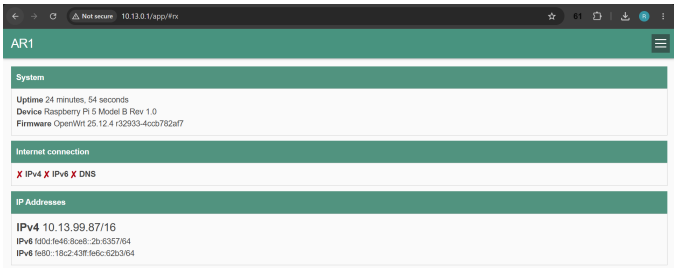

Fig. 22. LibreMesh management dashboard running on Raspberry Pi 5.

- Routing information

The successful operation of the management interface confirmed that the LibreMesh application framework, web services, and supporting middleware had been installed correctly.

Overall, the firmware exploration process demonstrated successful deployment of a customized LibreMesh image and verified that routing, wireless networking, management services, and BATMAN-adv components were operational prior to further mesh-network experimentation.

E. Mesh Validation and Connectivity Verification

Following deployment of two Raspberry Pi 5 nodes running LibreMesh, a series of connectivity tests were conducted to verify successful mesh formation and end-to-end communication. The nodes were interconnected using an Ethernet link configured as a BATMAN-adv transport interface. One Raspberry Pi hosted a wireless access point, while client devices including a laptop computer and smartphone connected to the network for testing.

Mesh operation was verified using BATMAN-adv diagnostic commands and Linux neighbor discovery tools. The command:

```
batctl meshif bat0 n
```

confirmed successful neighbor discovery between the two mesh nodes, indicating that BATMAN-adv was exchanging routing information across the Ethernet transport.

Additional verification was performed using:

```
ip neigh
```

which displayed dynamically learned neighboring devices throughout the mesh. The neighbor table contained entries corresponding to the participating Raspberry Pi nodes as well as connected client devices. These entries demonstrated that Layer 2 and Layer 3 communication was successfully propagating across the mesh infrastructure.

During testing, a smartphone was connected to the LibreMesh wireless network while a laptop computer maintained an SSH session into one of the Raspberry Pi nodes. The neighbor table showed reachable devices from multiple segments of the network, confirming successful forwarding of traffic through the mesh. ICMP testing further demonstrated successful communication between participating nodes.

These results confirmed that LibreMesh successfully established a functioning mesh topology using BATMAN-adv and automatically managed addressing within the 10.13.0.0/16 network space. The experiment validated neighbor discovery, route propagation, and client connectivity across the deployed mesh.

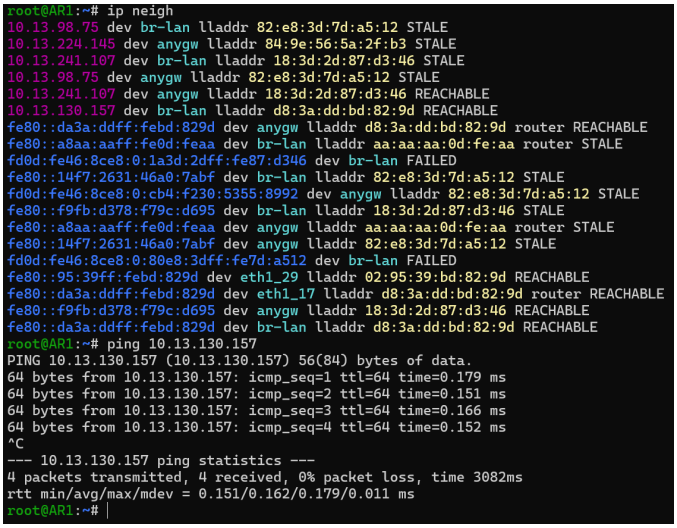

Fig. 23. Neighbor table showing dynamically learned devices within the LibreMesh deployment.

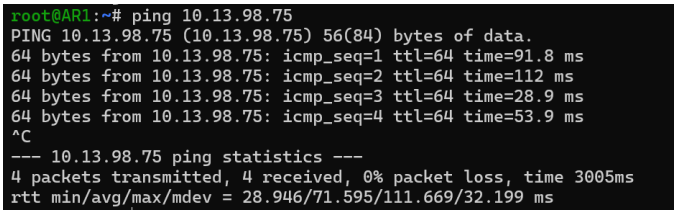

Fig. 24. Neighbor table showing dynamically learned devices within the LibreMesh deployment.

F. Internet Gateway Deployment and Validation

Following successful deployment of LibreMesh on Raspberry Pi 5 hardware, Internet gateway functionality was implemented using a dedicated Wide Area Network (WAN) interface. Initial attempts focused on providing Internet connectivity through wireless hotspot bridging; however, testing revealed that the Raspberry Pi 5 onboard wireless chipset could not reliably support simultaneous LibreMesh access point

services, mesh functionality, and wireless client operation. As a result, a dedicated Ethernet-based gateway architecture was adopted.

The LibreMesh network configuration was modified to remove the second Ethernet interface (`eth1`) from the default bridge configuration. This interface was then reconfigured as a DHCP WAN interface and connected directly to an upstream residential router. Upon activation, the node successfully obtained an IP address from the upstream network and installed a default route to the Internet.

A simplified topology of the resulting configuration is shown below:

```
Internet
|
Home Router
|
eth1 (WAN)
|
AR1
|
BATMAN-adv / LibreMesh
|
Wireless and Mesh Clients
```

Verification of Internet connectivity was performed using standard network diagnostic tools. The node successfully acquired the address `192.168.1.235` from the upstream router and established a default route through gateway `192.168.1.254`. Connectivity was confirmed by successfully pinging external Internet hosts, including Google's public DNS server (`8.8.8.8`).

LibreMesh AnyGW functionality was then enabled by configuring BATMAN-adv gateway mode as a server. Verification using BATMAN diagnostic tools confirmed that the node was advertising gateway services to participating mesh nodes.

To validate end-user connectivity, an Apple iPhone was connected to the LibreMesh wireless access point (`EC3730`). The device successfully received a LibreMesh network address within the mesh subnet and was able to access external Internet resources through the gateway node. Client association was verified using wireless station monitoring tools, while routing behavior was confirmed through neighbor table inspection and Internet connectivity testing.

This experiment demonstrated successful integration of LibreMesh routing, BATMAN-adv gateway services, wireless client access, and upstream Internet connectivity on physical Raspberry Pi hardware. The resulting configuration establishes the foundation required for future AnyGW testing involving multiple interconnected mesh nodes and automatic gateway discovery across the mesh network.

G. Wireless Mesh Interface Investigation and Hardware Limitations

During testing, an effort was made to utilize LibreMesh's native IEEE 802.11s wireless mesh functionality on Raspberry Pi 5 hardware. LibreMesh automatically generated a wireless mesh interface (`wlan0-mesh`) and associated BATMAN-adv configuration; however, the interface repeatedly failed to become operational despite appearing correctly configured within OpenWrt.

```
root@AR1:~# ifstatus wan
{
        "up": true,
        "pending": false,
        "available": true,
        "autostart": true,
        "dynamic": false,
        "uptime": 463,
        "l3_device": "eth1",
        "proto": "dhcp",
        "device": "eth1",
        "updated": [
                "addresses",
                "routes",
                "data"
        ],
        "metric": 0,
        "dns_metric": 0,
        "delegation": true,
        "ipv4-address": [
                {
                        "address": "192.168.1.235",
                        "mask": 24
                }
        ],
        "ipv6-address": [

        ],
        "ipv6-prefix": [

        ],
        "ipv6-prefix-assignment": [

        ],
        "route": [
                {
                        "target": "0.0.0.0",
                        "mask": 0,
                        "nexthop": "192.168.1.254",
                        "source": "192.168.1.235/32"
                }
        ],
        "dns-server": [
                "192.168.1.254"
```

Fig. 25. WAN interface successfully obtaining a DHCP address from the upstream router.

Several troubleshooting procedures were performed, including verification of wireless configuration files, inspection of kernel logs, interface status monitoring, and testing of simultaneous wireless access point and client operation. Although the mesh interface was present within the LibreMesh configuration, diagnostic commands consistently showed that the interface was unable to successfully initialize. Kernel logs repeatedly reported interface activation failures and MTU-related errors during wireless mesh initialization.

Further investigation was conducted using the command:

```
iw phy phy0 info
```

which revealed the wireless capabilities supported by the Raspberry Pi 5 onboard Broadcom wireless chipset. The supported interface modes included:

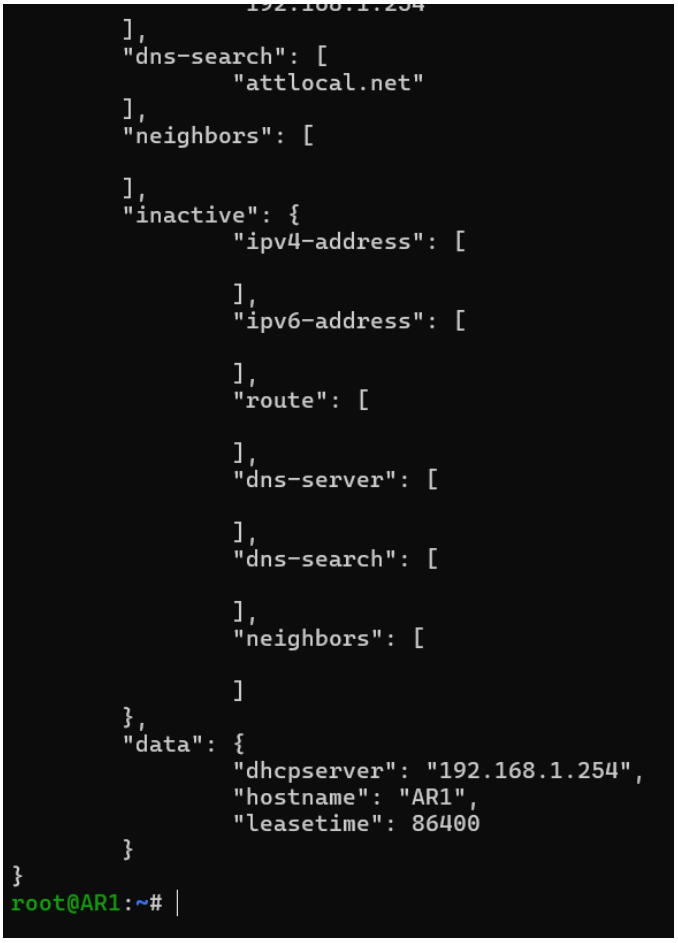

Fig. 26. WAN interface successfully obtaining a DHCP address from the upstream router.

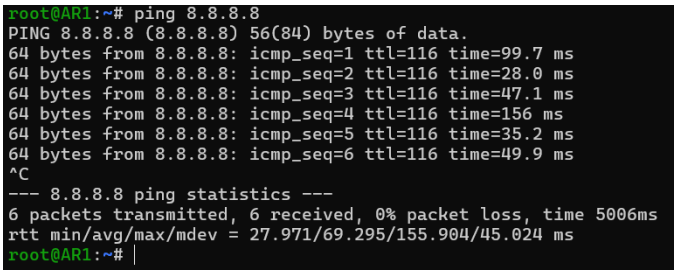

Fig. 27. Successful Internet connectivity verification from the LibreMesh gateway node.

- Managed (Client)
- Access Point (AP)
- IBSS (Ad-Hoc)
- Peer-to-Peer (P2P)

However, the wireless driver did not advertise support for the IEEE 802.11s `mesh point` interface mode required for native wireless mesh networking. As a result, LibreMesh was unable to establish a functional wireless mesh interface despite generating the appropriate configuration.

These findings suggest that the limitation originates from

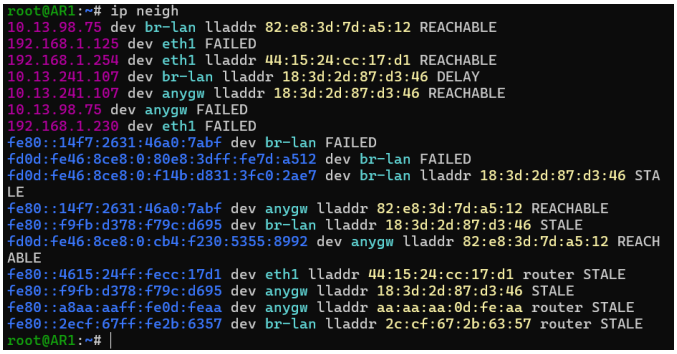

Fig. 28. Neighbor table showing a wireless client connected to the LibreMesh access point and participating in the mesh network.

the Broadcom `brcmfmac` driver and associated firmware rather than from LibreMesh itself. While wireless access point functionality operated correctly and BATMAN-adv routing functioned successfully over Ethernet links, native wireless mesh operation could not be validated on the Raspberry Pi 5 platform using the onboard wireless hardware.

Consequently, all successful mesh-network testing performed during this project utilized BATMAN-adv over Ethernet transport links between Raspberry Pi nodes. Wireless connectivity remained available for client devices through the LibreMesh access point service, allowing smartphones, laptops, and other devices to access the mesh network while routing functions were performed through wired backbone connections.

Future work may investigate the use of external USB wireless adapters with confirmed IEEE 802.11s support, alternative OpenWrt-compatible hardware platforms, or updated wireless drivers that provide native mesh-point functionality.

H. Demonstration Plan and Presentation Validation

A live demonstration is planned to validate the physical LibreMesh deployment and illustrate mesh-network operation using two Raspberry Pi 5 nodes configured with LibreMesh firmware. The two nodes will be interconnected using an Ethernet backbone connection through their mesh interfaces, allowing BATMAN-adv to establish neighbor relationships and exchange routing information.

One Raspberry Pi will be configured as an Internet gateway node. The gateway node will utilize a dedicated WAN interface connected to a laptop computer. The laptop will obtain Internet connectivity through a mobile hotspot connection and provide upstream network access to the LibreMesh node through Ethernet. This configuration will allow the LibreMesh gateway to distribute Internet access throughout the mesh network using LibreMesh AnyGW services.

A simplified demonstration topology is shown below:

```
Mobile Hotspot
|
Laptop
|
```

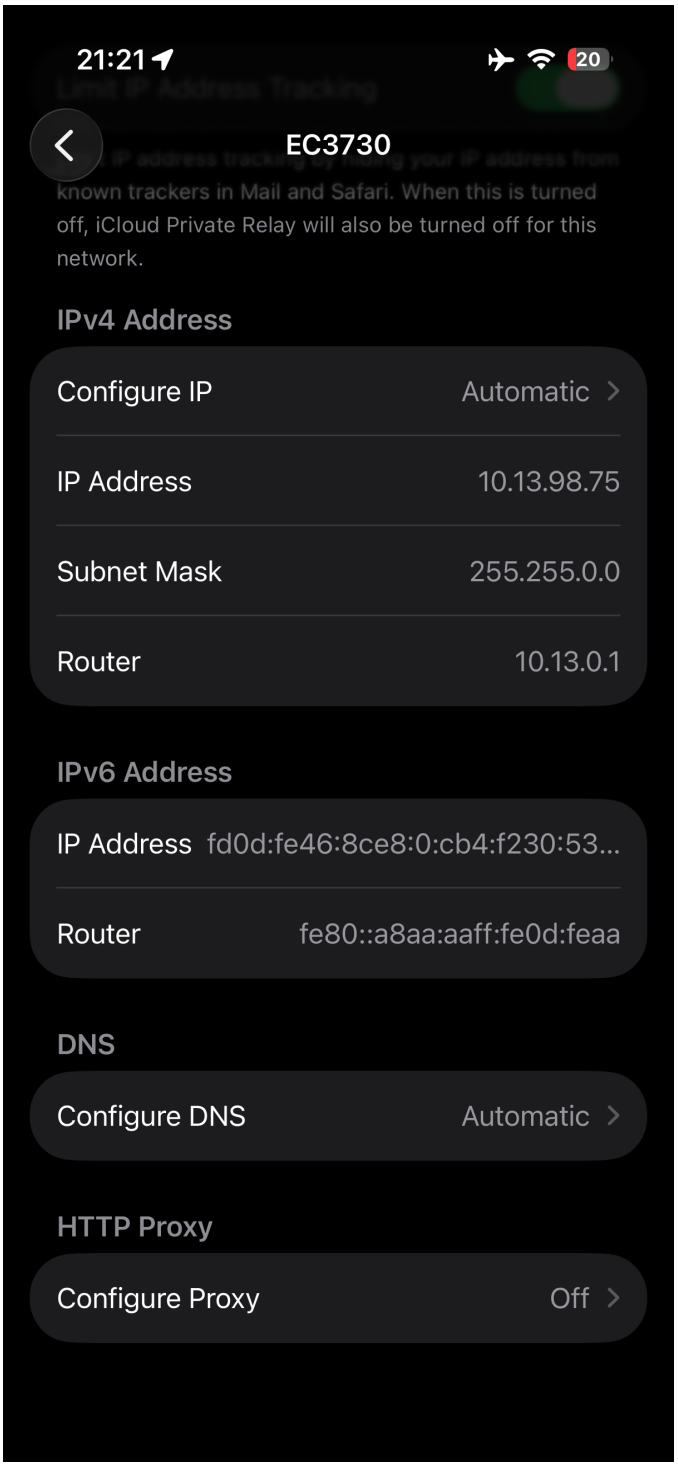

Fig. 29. Neighbor table showing a wireless client connected to the LibreMesh access point and participating in the mesh network.

```
root@AR1:~/ec3730/scripts# iw phy phy0 info | grep -A5 "Supported interface
modes"
        Supported interface modes:
                 * IBSS
                 * managed
                 * AP
                 * P2P-client
                 * P2P-GO
```

Fig. 30. Supported wireless interface modes reported by the Raspberry Pi 5 Broadcom wireless chipset. The absence of IEEE 802.11s mesh-point support prevented native wireless mesh operation.

```
Ethernet
|
AR1 (Gateway)
|
BATMAN-adv Mesh
|
BR1 Node
|
Wireless Clients
```

Audience members will be invited to join the EC3730 wireless network using smartphones, tablets, and laptop computers. Devices connected to the wireless network will receive LibreMesh IP addresses and gain access to both local mesh resources and external Internet connectivity through the gateway node.

The demonstration will focus on four primary validation activities:

1) Verification of BATMAN-adv neighbor discovery using:
   `batctl n`
   This command will display neighboring mesh nodes and confirm successful Layer 2 mesh formation across the Ethernet backbone.
2) Verification of route propagation and originator advertisements using:
   `batctl o`
   The originator table will demonstrate that routing information is being exchanged between mesh participants and that BATMAN-adv has established valid forwarding paths.
3) Verification of connected clients using:
   `ip neigh`
   `iw dev wlan0-ap station dump`
   These commands will display client devices currently participating in the mesh network and verify successful wireless association with the LibreMesh access point.
4) Verification of Internet connectivity and end-to-end communication.
   Connected users will be instructed to access Internet resources and perform ICMP connectivity tests. Participants will ping external hosts such as Google's public DNS server (`8.8.8.8`) and communicate with other connected devices to demonstrate successful routing through the mesh infrastructure.

Successful completion of these tests will demonstrate operation of LibreMesh services, BATMAN-adv routing, AnyGW

gateway functionality, wireless client access, and Internet connectivity within a physical mesh-network deployment. The presentation will provide a practical example of decentralized networking principles and illustrate how community mesh networks can distribute connectivity across multiple nodes while maintaining access to external network resources.

XIII. Unresolved Operational Challenges and Future Investigation

Although the distributed BATMAN-adv mesh deployment operated successfully overall, several operational inconsistencies and unresolved behaviors were identified during experimentation.

These issues represent valuable future investigation areas for both virtualized and physical mesh deployments.

A. Persistent VMware DHCP Address Duplication

Although earlier sequential startup procedures improved DHCP convergence behavior, VMware NAT services still occasionally assigned identical WAN-side addresses to multiple cloned OpenWrt nodes.

Example observed behavior included:

- Multiple nodes simultaneously assigned 192.168.111.141
- Ambiguous SSH access
- Non-deterministic LuCI management behavior
- Administrative-plane collisions

Although sequential startup reduced the frequency of collisions, duplicate assignments still occasionally reappeared during extended experimentation.

The issue appeared isolated to VMware NAT-side DHCP behavior and did not affect BATMAN forwarding functionality operating on the dedicated mesh backbone interface.

This behavior represents an important virtualization-management limitation requiring further investigation.

B. IPv6 Duplicate Address Detection Events

OpenWrt repeatedly reported IPv6 duplicate address detection warnings during sequential startup and node cloning operations:

```
IPv6 duplicate address detected
```

Despite repeated warnings:

- IPv4 routing remained operational
- BATMAN mesh convergence persisted
- Node-to-node communication remained stable

The project did not fully investigate:

- IPv6 address autoconfiguration behavior
- Duplicate IPv6 neighbor advertisements
- IPv6 interactions with BATMAN-adv
- Potential virtualization-layer MAC/address reuse

Future work should evaluate whether disabling IPv6 entirely or implementing static IPv6 addressing would improve deployment stability.

C. Management-Plane and Forwarding-Plane Separation

The project revealed significant architectural separation between:

- Administrative management-plane behavior
- BATMAN distributed forwarding behavior

Even during severe management-plane inconsistencies:

- BATMAN neighbor discovery remained stable
- Originator propagation continued
- Distributed forwarding functionality persisted

This behavior further confirmed the resilience of BATMAN forwarding despite administrative-plane instability.

D. Lightweight Embedded Linux Limitations

Several OpenWrt operational limitations were identified during experimentation:

- Missing HTTPS support in BusyBox wget
- Missing SFTP subsystem support in Dropbear
- Minimal shell-tool availability
- CRLF/LF shell-script compatibility issues

These limitations complicated:

- Package installation
- Secure artifact transfer
- Deployment automation
- Remote administration

While expected for lightweight embedded environments, these limitations introduced significant operational overhead during deployment and experimentation.

E. Incomplete Multi-Hop Routing Validation

Although the BATMAN mesh operated successfully across all four nodes, the VMware host-only topology allowed all routers to remain directly adjacent at Layer 2.

As a result, the deployment did not fully validate:

- True multi-hop relay behavior
- Intermediate-node forwarding dependency
- Mesh partition recovery
- Indirect route optimization

Future work should intentionally isolate subsets of nodes to force multi-hop forwarding paths and evaluate true relay-based mesh behavior.

F. Future WireGuard-Based Inter-Mesh Federation

Future experimentation may connect independent BATMAN mesh deployments between project groups using encrypted WireGuard overlay tunnels.

Potential future architecture:

- Independent local BATMAN clusters
- WireGuard encrypted Internet overlays
- Distributed inter-group route propagation
- Geographically separated mesh federation

This would allow evaluation of:

- Distributed mesh scalability
- Internet-based overlay transport
- Secure inter-cluster communication
- Hybrid mesh-overlay architectures

XIV. Conclusion

This project successfully implemented a persistent four-node distributed OpenWrt mesh-routing laboratory environment using VMware virtualization and BATMAN-adv.

The completed deployment demonstrated:

- Successful OpenWrt virtualization
- Stable multi-node routed mesh backbone construction
- Autonomous BATMAN neighbor discovery
- Dynamic distributed originator propagation
- Persistent mesh deployment through UCI configuration
- Autonomous reconvergence after node failure
- BATMAN-native ping and traceroute functionality
- MTU optimization and fragmentation analysis
- Packet-level mesh observation using tcpdump
- Identification of virtualization and management-plane edge cases

The project evolved significantly beyond initial router virtualization objectives and ultimately produced a functional distributed autonomous mesh-routing research platform suitable for continued LibreMesh experimentation and future Raspberry Pi deployment.

The completed environment represents a robust foundation for continued experimentation with decentralized community-networking infrastructure using LibreMesh and BATMAN-adv.

A. Post Presentation Conclusion

This project successfully implemented and evaluated LibreMesh and BATMAN-adv in both virtualized and physical deployment environments. Initial experimentation was conducted using a four-node OpenWrt mesh laboratory hosted within VMware, providing a controlled environment for studying decentralized routing behavior, fault tolerance, route propagation, and mesh-network management. The project was subsequently extended to physical Raspberry Pi 5 hardware, where LibreMesh was deployed and validated using real wireless clients and Internet gateway services.

The completed deployment demonstrated successful OpenWrt virtualization, BATMAN-adv mesh formation, autonomous neighbor discovery, dynamic route propagation, persistent configuration management through UCI, and recovery from topology changes. Additional testing validated wireless client connectivity, Internet gateway functionality, and operation of LibreMesh AnyGW services. A dedicated WAN interface was successfully implemented on a Raspberry Pi node, allowing Internet connectivity to be distributed to wireless clients connected through the mesh network.

A key outcome of this project was understanding the interaction between multiple routing technologies within LibreMesh. BATMAN-adv operates primarily as a Layer 2 mesh-routing protocol, creating a virtual distributed Ethernet segment across participating nodes. Unlike traditional routing protocols such as OSPF, BATMAN-adv does not maintain a complete network topology database or compute shortest paths using Dijkstra's algorithm. Instead, nodes exchange Originator Messages (OGMs) and determine only the best next-hop neighbor toward each destination. This approach reduces routing overhead and improves adaptability in dynamic mesh environments.

LibreMesh additionally incorporates the Babel routing protocol, which operates at Layer 3 and is responsible for exchanging IP routing information between nodes. While BATMAN-adv provides distributed Layer 2 connectivity, Babel contributes Layer 3 route dissemination and network reachability information. Together, these protocols provide a hybrid architecture that combines Layer 2 mesh forwarding with Layer 3 routing intelligence.

Internet gateway distribution is facilitated through LibreMesh AnyGW functionality. AnyGW allows nodes possessing Internet connectivity to advertise gateway services throughout the mesh. Other mesh participants may automatically discover and utilize available gateways without requiring manual route configuration. Testing demonstrated successful operation of a gateway-enabled Raspberry Pi node and validated Internet access from wireless client devices connected to the mesh.

The project also revealed several practical implementation considerations. Configuration files stored by OpenWrt and LibreMesh maintain wireless SSIDs and pre-shared keys in plain-text format within system configuration files rather than storing only cryptographic hashes. While this simplifies administration and automated deployment, it also highlights the importance of securing administrative access to mesh nodes and protecting configuration backups from unauthorized disclosure.

Overall, the completed environment represents a functional distributed mesh-networking platform suitable for continued experimentation with decentralized community networking, Internet gateway distribution, routing protocol analysis, and future extensions involving WireGuard integration, larger mesh deployments, and advanced LibreMesh services. The project successfully demonstrated both the theoretical and practical aspects of modern mesh networking and established a robust foundation for future research and development.

References

[1] OpenWrt Project, "OpenWrt Documentation," [Online]. Available: https://openwrt.org

[2] LibreMesh Project, "LibreMesh Documentation," [Online]. Available: https://libremesh.org

[3] BMX7 Routing Protocol, "BMX7 GitHub Repository," [Online]. Available: https://github.com/bmx-routing/bmx7

[4] BATMAN-adv Project, "B.A.T.M.A.N. advanced Documentation," [Online]. Available: https://www.open-mesh.org/projects/batman-adv/wiki

[5] Open-Mesh, "batctl User Guide," [Online]. Available: https://www.open-mesh.org/projects/batman-adv/wiki/Batctl

[6] VMware Inc., "VMware Workstation Pro Documentation," [Online]. Available: https://docs.vmware.com

[7] OpenAI, "ChatGPT," [Online]. Available: https://chat.openai.com

[8] LibreMesh Project, "LibreMesh Documentation," [Online]. Available: https://libremesh.org

[9] LibreMesh Project, "Getting Started," [Online]. Available: https://libremesh.org/getting-started.html

[10] LibreMesh Project, "Network Protocol Options," [Online]. Available: https://libremesh.org/reference/network/protocols-options.html

[11] Earth Defenders Toolkit, "Guide: LibreRouter and LibreMesh," [Online]. Available: https://www.earthdefenderstoolkit.com/one-pager/guide-librerouter-and-libremesh/
[12] BATMAN-adv Project, "B.A.T.M.A.N. advanced Documentation," [Online]. Available: https://www.open-mesh.org/doc/batman-adv/Wiki.html
[13] OpenWrt Project, "B.A.T.M.A.N. / batman-adv," [Online]. Available: https://openwrt.org/docs/guide-user/network/wifi/mesh/batman
[14] OpenWrt Project, "Security," [Online]. Available: https://openwrt.org/docs/guide-developer/security
[15] OpenWrt Project, "opkg update Signature check failed for luci," GitHub Issue, [Online]. Available: https://github.com/openwrt/openwrt/issues/19316
[16] R. Baig, R. Roca, F. Freitag, and L. Navarro, "guifi.net, a crowdsourced network infrastructure held in common," Computer Networks, 2015.
[17] L. Cerda-Alabern, "Experimental Evaluation of a Wireless Community Mesh Network," [Online]. Available: https://pdfs.semanticscholar.org/e18f/9cff9dab586d63c54149014ec99f2eaea134.pdf
[18] Gluon Project, "Welcome to Gluon," [Online]. Available: https://gluon.readthedocs.io/
[19] Resilient Comms, "NYC Mesh: Community Internet Infrastructure," [Online]. Available: https://resilientcomms.org/case-studies/nyc-mesh-community-network
[20] IEEE Denver, "Securing Wireless Mesh Networks," [Online]. Available: https://ieee-denver.org/wp-content/uploads/sites/23/2022/10/Securing_Wireless_Mesh_Networks-1.pdf
[21] Open Technology Fund, "Commotion Android Security Audit," [Online]. Available: https://www.opentech.fund/security-safety-audits/commotion-android/
[22] LibreMesh Project, *Network Protocol Options*, Available: https://libremesh.org/reference/network/protocols-options.html Accessed: May 2026.
[23] LibreMesh Project, *How LibreMesh Works*, Available: https://www.onnocenter.or.id/wiki/index.php/LibreMesh:_How_It_Works Accessed: May 2026.

AI Usage Disclosure

Artificial intelligence tools were used during portions of this project to assist with:

- Technical troubleshooting
- Linux and OpenWrt command analysis
- BATMAN-adv deployment guidance
- VMware networking diagnostics
- LaTeX report formatting
- Documentation organization and editing

AI-assisted guidance was used interactively throughout deployment and experimentation; however, all configuration, implementation, troubleshooting, validation, testing, and analysis were performed directly by the project team.

The report content was reviewed, modified, and validated manually to ensure technical accuracy and consistency with observed system behavior.

OpenAI ChatGPT was used as an engineering-assistance and documentation-support tool during the project lifecycle.

Appendix A
Exported Configuration and Operational Artifacts

The following exported artifacts were generated and archived from each OpenWrt node:

- network-nodeX
- system-nodeX
- firewall-nodeX
- dhcp-nodeX
- batman-if-nodeX.txt
- batman-neighbors-nodeX.txt
- batman-originators-nodeX.txt
- ip-addr-nodeX.txt
- ip-route-nodeX.txt
- packages-nodeX.txt

These artifacts provide reproducible deployment state information and operational validation for the completed distributed BATMAN-adv mesh environment.

Appendix

This appendix documents the primary commands used throughout the project during virtual machine development, Git repository management, LibreMesh deployment, BATMAN-adv configuration, AnyGW implementation, and system validation.

A. VMware and Windows Commands

1) Network Configuration:

a) Display Network Configuration:

```
ipconfig
```

Displays the IP configuration of all network adapters including IPv4 addresses, subnet masks, and default gateways.

b) Display Detailed Network Configuration:

```
ipconfig /all
```

Displays detailed information about all network adapters, including MAC addresses, DHCP information, DNS servers, and lease information.

c) Ping a Device:

```
ping <IP Address>
```

Tests network connectivity to another device by sending ICMP echo requests.

Example:

```
ping 10.13.99.87
```

d) Display ARP Table:

```
arp -a
```

Displays the Address Resolution Protocol (ARP) cache showing IP-to-MAC address mappings.

2) SSH Connectivity:

a) Connect to a LibreMesh Node:

```
ssh root@<IP Address>
```

Creates a secure shell connection to a LibreMesh node.

Example:

```
ssh root@10.13.99.87
```

3) Git Repository Management:

a) Check Repository Status:

```
git status
```

Displays modified, staged, and untracked files.

b) Add All Files:

```
git add .
```

Stages all modified files for commit.

c) Create Commit:

```
git commit -m "Commit Message"
```

Creates a snapshot of the repository changes.

d) Upload Changes:

```
git push
```

Uploads local commits to the remote repository.

e) Clone Repository:

```
git clone <repository-url>
```

Downloads a repository from a remote source.

f) Pull Updates:

```
git pull
```

Downloads and merges updates from the remote repository.

g) Remove Cached File:

```
git rm --cached <filename>
```

Removes a file from Git tracking while keeping the local copy.

h) Remove All Cached Files:

```
git rm -r --cached .
```

Removes all files from Git tracking without deleting local copies.

B. Raspberry Pi and LibreMesh Commands

1) Wireless Configuration:

a) Display Wireless Configuration:

```
uci show wireless
```

Displays all wireless settings stored in OpenWrt.

b) Display Wireless Interfaces:

```
iw dev
```

Displays all wireless interfaces and operating modes.

c) Display Wireless Information:

```
iwinfo
```

Displays SSID, channel, encryption, signal levels, and interface status.

d) Scan Nearby Wireless Networks:

```
iw dev wlan0-mesh scan | grep SSID
```

Scans for nearby wireless networks and displays detected SSIDs.

e) Reload Wireless Configuration:

```
wifi reload
```

Applies wireless configuration changes without requiring a reboot.

2) Network Configuration:

a) Display Network Configuration:

```
uci show network
```

Displays all OpenWrt network settings.

b) Display Interface Status:

```
ifstatus <interface>
```

Displays operational information about a network interface.
Examples:

```
ifstatus wan
ifstatus wwan
```

c) Display IP Addresses:

```
ip addr
```

Shows all network interfaces and assigned IP addresses.

d) Display Routing Table:

```
ip route
```

Displays the kernel routing table.

e) Display Neighbor Table:

```
ip neigh
```

Displays neighboring devices discovered through ARP and IPv6 Neighbor Discovery.

f) Restart Networking:

```
/etc/init.d/network restart
```

Restarts all OpenWrt networking services.

3) UCI Configuration Commands:

a) Read Configuration Value:

```
uci get <option>
```

Displays the current value of a configuration option.
Example:

```
uci get network.bat0.gw_mode
```

b) Modify Configuration:

```
uci set <option>=<value>
```

Changes a configuration setting.
Example:

```
uci set wireless.lm_wlan0_ap_radio0.ssid='EC3730A'
```

c) Delete Configuration Entry:

```
uci delete <section>
```

Removes a configuration section.
Example:

```
uci delete wireless.wifinet3
```

d) Commit Configuration Changes:

```
uci commit
```

Writes configuration changes to disk.

e) Commit Wireless Changes:

```
uci commit wireless
```

Writes wireless configuration changes to disk.

4) BATMAN-adv Commands:

a) Display Neighbor Table:

```
batctl n
```

Displays neighboring BATMAN-adv mesh nodes.

b) Display Originator Table:

```
batctl o
```

Displays routing information exchanged between mesh nodes.

c) Display Gateway List:

```
batctl gwl
```

Displays available BATMAN gateway servers.

d) Display Gateway Mode:

```
batctl gw_mode
```

Displays whether the node is operating as a gateway client or server.

5) Connectivity Testing:

a) Ping External Host:

```
ping 8.8.8.8
```

Tests Internet connectivity using Google's public DNS server.

b) Ping Domain Name:

```
ping google.com
```

Tests Internet connectivity and DNS resolution.

6) Bridge Verification:

a) Display Bridge Configuration:

```
brctl show
```

Displays Linux bridge interfaces and associated ports.

7) Client Verification:

a) Display Connected Wireless Clients:

```
iw dev wlan0-ap station dump
```

Displays wireless clients currently associated with the access point.

8) LibreMesh Validation Scripts:

a) Connectivity Validation:

```
./test_connectivity.sh
```

Custom script used to verify network connectivity and LibreMesh status.

b) Mesh Validation:

```
./check_mesh.sh
```

Custom script used to display mesh interfaces, neighbors, routes, and wireless information.

9) Thermal Monitoring:

a) Display CPU Temperature:

```
cat /sys/class/thermal/thermal_zone0/temp
```

Displays Raspberry Pi CPU temperature in millidegrees Celsius.

b) Display Cooling Device Type:

```
cat /sys/class/thermal/cooling_device0/type
```

Displays the active cooling device driver.

c) Display Current Fan State:

```
cat /sys/class/thermal/cooling_device0/cur_state
```

Displays the current fan speed state.

d) Display Maximum Fan State:

```
cat /sys/class/thermal/cooling_device0/max_state
```

Displays the maximum supported fan speed state.

e) Search for Fan Events:

```
dmesg | grep -i fan
```

Searches kernel logs for fan-related events.

f) Display PWM Fan Controls:

```
ls /sys/class/hwmon/hwmon*/pwm*
```

Lists available PWM fan control interfaces.

10) AnyGW Gateway Configuration:

a) Remove Interface from LAN Bridge:

```
uci del_list network.@device[0].ports='eth1'
```

Removes the second Ethernet interface from the LibreMesh bridge.

b) Create WAN Interface:

```
uci set network.wan='interface'
uci set network.wan.device='eth1'
uci set network.wan.proto='dhcp'
```

Creates a dedicated WAN interface using DHCP.

c) Enable BATMAN Gateway Server Mode:

```
uci set network.bat0.gw_mode='server'
uci commit network
/etc/init.d/network restart
```

Configures the node to advertise itself as an Internet gateway to mesh participants.